A university canteen operates a self-service kiosk that allows students to (1) calculate the total cost of a meal order, (2) compute a weekly nutrition summary, and (3) enter daily calorie readings until done. You are hired to implement the kiosk software in C++. All four questions below relate to this system.

Question 1 · Menu-Driven Program Using `switch`

[30 marks]

1(a) — Main kiosk menu 5 marks

Write a complete C++ program that displays the menu below and uses a `switch` statement to execute the selected option. The program must loop until the user chooses option 4 (Exit).

```
===== CANTEEN KIOSK ===== 1. Calculate meal cost 2. View nutrition summary 3. Enter calorie log 4. Exit Enter choice: _
```

For options 1–3, print a placeholder message (e.g. `"Option 1 selected."`). For any invalid input print: `Invalid option. Please try again.`

No marks will be awarded for this part if `if-else` is used instead of `switch`.

1(b) — Nested switch for meal pricing 8 marks

Extend option 1 so that selecting "Calculate meal cost" displays a secondary meal-category menu:

```
-- Meal Category -- A. Main meal (R 45.00) B. Snack (R 18.00) C. Beverage (R 12.00) D. Combo deal (R 65.00) Enter category: _
```

Use a **nested `switch` on the `char` input** (accept both upper- and lower-case). Declare all prices as `const double` constants. After the category is chosen, prompt for a quantity (1–10). Validate the quantity with a loop and reject values outside this range. Display the line total formatted to 2 decimal places.

Sample output for category B, quantity 3:

```
Item : Snack Qty : 3 Total : R 54.00
```

1(c) — Meal plan discount with switch 7 marks

After the line total is computed in 1(b), the kiosk must apply a **meal plan discount** based on the student's tier. Prompt the user to enter their tier number (1, 2, or 3) and use a **further `switch` statement** to apply the following discounts:

TIER	PLAN NAME	DISCOUNT
1	Gold	20% off the line total
2	Silver	10% off the line total
3	Bronze	5% off the line total

If an invalid tier is entered, display an error and apply **no discount** (0%). Display a full payment summary as shown below. Use `setprecision(2)` and `fixed` throughout.

```
========== Payment Summary ========== Item : Combo deal Quantity : 2 Subtotal : R 130.00 Tier : Silver (10% off) Discount : R 13.00 Amount due : R 117.00
=====================================
```

10-mark deduction rule: The entire Question 1 solution must use `switch` exclusively for all branching decisions. Any use of `if-else` for menu or tier selection will attract a 10-mark penalty.

1(d) — Short answer 10 marks

Answer the following questions in 2–3 sentences each:

#	QUESTION	MARKS
i	Explain why a `break` statement is required at the end of each `case` in a `switch`. What happens if it is omitted? Illustrate with a one-line example.	3
ii	What is the role of the `default` case in a `switch` statement? Is it mandatory? Justify your answer with reference to the kiosk menu.	3

iii	State **two limitations** of the `switch` statement compared with `if-else if` chains. Give a kiosk-related example where `switch` cannot be used and `if-else` must be used instead.	4

Question 2 · Counter-Controlled Loops for Sum Calculations

2(a) — Weekly calorie expenditure 8 marks

The canteen records a student's daily calorie intake for each of the 7 days of the week. Write a C++ program that:

Step 1 Uses a counter-controlled `for` loop to read exactly 7 calorie values (each ≥ 0)
Step 2 Computes the weekly total and daily average (2 d.p.)
Step 3 Identifies the highest and lowest daily intake within the same loop
Step 4 Displays all four results in a formatted report

You must compute all four values using a **single `for` loop** — no additional loops for min/max.
Sample output:

```
Day 1 calories: 2100 Day 2 calories: 1850 Day 3 calories: 2300 Day 4
calories: 1950 Day 5 calories: 2450 Day 6 calories: 1600 Day 7 calories: 2200 =====
Weekly Nutrition Report ===== Weekly total : 14450 kcal Daily average : 2064.29
kcal Highest day : 2450 kcal Lowest day : 1600 kcal
===================================
```

2(b) — Nutrition score table 9 marks

The canteen assigns a *nutrition score* to each of its **N** menu items (where N is entered by the user, 2 ≤ N ≤ 8). Each score is an integer 1–10. Write a program that uses **nested counter-controlled loops** to:
Outer loop Iterate over each of the N menu items
Inner loop Read 5 weekly ratings per item and compute that item's average score
After loops Print a formatted table and identify the item with the best average

Display each item's average right-aligned in a field of width 8 (2 d.p.) using `setw` and `fixed`. Sample output for N = 3:

```
Item 1 ratings: 7 8 6 9 7 Item 2 ratings: 5 6 5 7 6 Item 3 ratings: 9 10 8 9
10 Item Average -------------------- Item 1 7.40 Item 2 5.80 Item 3 9.20 ----------
---------- Best item : Item 3 (avg 9.20)
```

2(c) — Short answer 8 marks

#	QUESTION	MARKS
i	Define a **counter-controlled loop**. State the three essential components of a `for` loop and explain the role of each in the context of reading 7 daily calorie values.	3
ii	What is a **nested loop**? State the formula for the total number of iterations when an outer loop runs *M* times and an inner loop runs *K* times. Apply the formula to 2(b) with N = 4 items and 5 ratings each.	3
iii	Rewrite the following `for` loop as an equivalent `while` loop. Is there any situation in this kiosk context where you would prefer `while` over `for`? Explain. `for (int d = 1; d <= 7; d++) sum += intake[d];`	2

Question 3 · Sentinel-Controlled Loop for Data Input

[25 marks]

3(a) — Daily calorie log with sentinel 13 marks

This implements the kiosk's "Enter calorie log" option (option 3 from Q1). Write a complete C++ program that uses a **sentinel-controlled `while` loop** to accept a student's daily calorie readings. The loop must terminate when the student enters **−1** (the sentinel value).
For each reading entered (before the sentinel), the program must:

ValidateReject values outside 0–9999 kcal with an inline error and re-prompt (use a nested loop)
AccumulateMaintain a running total and count of valid readings
ClassifyCount readings into Low (<1500), Normal (1500–2500), High (>2500)

After the sentinel is entered, display the full summary below. Guard against division by zero if no valid readings were entered.

```
====== Calorie Log Entry ====== Enter daily calories (-1 to finish): 2100 Enter
daily calories (-1 to finish): -50 ** Invalid. Must be 0 - 9999. Re-
enter: 1850 Enter daily calories (-1 to finish): 3100 Enter daily calories (-1 to
finish): 800 Enter daily calories (-1 to finish): -1 ======== Calorie Summary
======== Entries recorded : 4 Total calories : 7850 kcal Daily average : 1962.50
kcal --------------------------------- Low (<1500) : 1 day(s) Normal (1500-2500): 2
day(s) High (>2500) : 1 day(s) ==================================
```

3(b) — Running average with do-while7 marks

Modify 3(a) so that after each valid calorie entry, the program immediately prints an updated running average. Use a **`do-while` loop** for the outer entry loop instead of a `while` loop. The program must still terminate on −1.
Sample interaction (partial):

```
Enter daily calories (-1 to finish): 2000 Running avg : 2000.00 kcal Enter daily
calories (-1 to finish): 1800 Running avg : 1900.00 kcal Enter daily calories (-1
to finish): 2400 Running avg : 2066.67 kcal Enter daily calories (-1 to finish): -1
```

Hint: with `do-while`, ensure the sentinel check happens *before* updating the accumulator — use an `if` guard inside the loop body.

3(c) — Short answer5 marks

#	QUESTION	MARKS
i	Explain what a **sentinel value** is. Why was −1 chosen as the sentinel for calorie input? Could 0 be used instead — justify your answer.	2
ii	Compare `while` and `do-while` loops: state one scenario in this kiosk system where a `do-while` is more appropriate than a `while`, and explain why.	3

Question 4 · Debug Loop Errors

[20 marks]

4(a) — Identify and correct six bugs12 marks · 2 each

The program below is a flawed attempt to implement parts of the canteen kiosk. It contains **exactly six loop-related bugs**, each marked `// BUG N`. For each bug: **(i)** name the error type, **(ii)** explain why it is wrong in this context, and **(iii)** write the corrected line. Then rewrite the fully corrected program at the end.

```
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    const int DAYS = 7;
    int    total = 0, calories, count = 0;
    double average;

    // — Part A: sum calories for 7 days —
    for (int d = 1; d < DAYS; d++) {        // BUG 1
        cout << "Day " << d << ": ";
        cin  >> calories;
        total += calories;
    }

    // — Part B: sentinel loop for extra entries —
```

```
    cout << "Enter calories (-1 to stop): ";
    while (calories != -1) {                // BUG 2
        if (calories >= 0 && calories <= 9999)
            count--;                        // BUG 3
        cout << "Enter calories (-1 to stop): ";
        cin >> calories;
    }

    // ── Part C: compute average ──
    average = total / count;                // BUG 4
    cout << "Average: " << average << endl;

    // ── Part D: nested loop — 4-day × 3-meal grid ──
    for (int day = 1; day <= 4; day++) {
        for (int meal = 1; meal <= 3; day++) {   // BUG 5
            cout << setw(6) << day * meal;
        }
        cout << " ";                          // BUG 6
    }

    return 0;
}
```

Record your answers in this table:

BUG #	ERROR TYPE	WHY IT IS WRONG	CORRECTED LINE
1			
2			
3			
4			
5			
6			

4(b) — Predict the output 4 marks

Without running it, write the exact output produced by the following code segment. Show your working.

```
int x = 10;
while (x > 0) {
    if (x % 3 == 0)
        cout << x << " ";
    x -= 2;
}
```

4(c) — Short answer 4 marks

#	QUESTION	MARKS
i	Define an **off-by-one error** and explain how Bug 1 in 4(a) is an instance of it. How can a programmer avoid this error when writing counter-controlled loops?	2
ii	Explain the concept of an **infinite loop**. Identify which bug in 4(a) could cause an infinite loop under certain conditions and justify your answer.	2